

Transaction and Concurrency Control

Unit: 4

DBMS BCSE0402

Course Details
(B.Tech IVth Sem)



Rashmi Bhardwaj
Assistant Professor
CSE Department

Syllabus

 NIET Greater Noida Autonomous Institute	NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY GREATER NOIDA-201306 (An Autonomous Institute) School of Computer Science & Information Technology
---	--

Course Code: BCSE0402		Course Name: Database Management Systems										L	T	P	C
Course Offered in: CSE/IT/CSE (Twin) /IT(Twin)/CSE(Prof)/IT(Prof)/M&C/AI/AI(TWIN)/AIML/AIML(TWIN)/ CS/CYS/DS/IOT												3	0	0	3
Pre-requisite: Basic understanding of computer fundamentals such as architecture, storage, and hardware. In addition, familiarity with data structures, algorithms, and basic programming concepts will be beneficial.															
Course Objectives: The objective of the course is to introduce about database management systems, with an emphasis on how to organize, maintain and retrieve - efficiently, and effectively - information in relational & non-relational databases.															
Course Outcome- After completion of this course students will be able to														Bloom's Knowledge Level (KL)	
CO 1	Apply ER model for conceptual design of the database.													K3	
CO2	Execute SQL and apply the normalization to improve the database design.													K3	
CO3	Implement complex queries in database with different applications.													K3	
CO4	Execute the concept of PL/SQL, transaction and concurrency control.													K3	
CO5	Evaluate and implement Relational and Non-Relational databases using different tools and their effectiveness in real-world applications.													K5	
CO-PO Mapping															
CO-PO Mapping	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3	PSO4

Syllabus

CO1	3	3	3	3	2	1	1	1	2	1	2	3	1	2	1
CO2	3	2	3	3	2	2	1	2	1	2	2	3	2	2	1
CO3	3	3	2	3	3	2	-	2	1	-	2	3	3	2	1
CO4	3	2	2	2	2	2	1	-	1	1	1	3	3	2	2
CO5	2	2	2	2	3	2	1	-	1	2	2	3	3	2	2
Course Contents / Syllabus															
Module 1		Introduction about the Database Conceptual Designing												8 hours	
Basic Concept: Database system concept, architecture, History of Database, Data Independence, Database system Vs File system, Data models & Types of Data Models, schema and instances. Data Modelling using the Entity Relationship Model: ER model concepts, Degree of relationship, Notation for ER diagram, mapping constraints, Generalization, Aggregation, Reduction of an ER diagrams to tables. Extended ER Diagram & reduction of EER. Codd Rules. Introduction on SQL: Types of SQL commands: -DDL, DML, DCL, TCL. Basic of Relation Algebra & Operations, Query Optimization.															
Module 2		Basic of SQL & Normalization												8 hours	
Keys & Types of Keys: Super key, Candidate Key, Primary Key, Alternative Key, Foreign Key, unique. Constraints and Types of Constraints. Use of Functions, Clause and Predicates: Aggregate Function, Scalar Functions, Where, Group by, Having and Order by, SQL Operators. Like, Between, Aliases, distinct, limit. Normalization: Functional Dependencies, attribute Closure, Normalization & Types of Normalization, Candidate Key, Canonical Cover of FD's.															
Module 3		Introduction of Complex Queries												8 hours	
Use of Operators: Union, Intersect, Minus, Cartesian Product, join:-Inner Join: - Natural Join, Equi Join & Non Equi Join Outer Join: Left Outer Join, Right Outer Join and Full Outer Join, Division Operator. Nested Query or Sub Query: IN, NOT IN, Exists, Not Exists, All and Any. Managing Indexes, Synonyms and Sequences, Managing Views. Introduction of PL/SQL: Implementation of PL/SQL Function, Procedure, Trigger, Cursor. Database connectivity: Database Connectivity with Java/Python Programming Languages.															
Module 4		Transaction and Concurrency Control												8 hours	
Transaction system: Life cycle of transaction, ACID Properties Schedule & Types of Schedule, Serializability, Recoverability, Deadlock Handling. Concurrency Control Techniques: Concurrency Control, Concurrency control Techniques: Locking Techniques, Timestamping, Validation Based Protocol, Transaction & Data Control: -Grant, Revoke, commit & Rollback.															
Module 5		Introduction of NoSQL With MongoDB												8 hours	
Introduction of NoSQL With MongoDB : Introduction of NoSQL Data Models, Overview of NoSQL Databases with their Types, Uses & Features of NoSQL Document Databases, CAP theorem, BASE Vs ACID, Comparison of relational databases to NoSQL stores, uses and deployment; - MongoDB, Cassandra, HBASE, Neo4j and Riak. Introduction and Features of MongoDB, MongoDB Operators, MongoDB Collection & Document, MongoDB Shell & their commands, CRUD operations. Cloud DatabaseIntroduction of Cloud Database. MongoDB Cloud product : Stitch, Atlas & Cloud Manager.															

Syllabus

		Total Lecture Hours	40
Textbook:			
S.No	Book Title		
1	Abraham Silberschatz, Henry Korth and S. Sudarshan, “Database Concepts”, McGraw Hill, 7th Edition, 2020		
2	Elmasri, Navathe, “Fundamentals of Database Systems”, Addison Wesley, 7 th edition, 2016		
Reference Book			
S.No	Book Title		
1	Thomas Cannolly and Carolyn Begg, Database Systems: A practical Approach to Design, Implementation and Management. Pearson Education,3rd Edition, 2007.		
2	Ted Hills, NoSQL and SQL Data Modeling: Bringing Together Data, Semantics, and Software, Ted Hills, 1st Edition,2016.		
NPTEL/ Youtube/ Faculty Video Link:			
Unit 1:	DBMS L1 Inauguration & Introduction (youtube.com) DBMS L2 Introduction to Relational Model (youtube.com) DBMS L3 Introduction to SQL (youtube.com) DBMS L8C Entity Relationship Model (youtube.com) DBMS L8D Entity Relationship Model (Problem Solving and Discussion) (youtube.com)		
Unit 2:	DBMS L4A Joins, Set Operations and Aggregate Functions (youtube.com) DBMS L9A Relational Database Design - YouTube DBMS L9B Relational Database Design (youtube.com) DBMS L9C Relational Database Design (youtube.com) DBMS L9D Discussion on Normalization (youtube.com) Relational Data Model and Notion of Keys - YouTube Introduction to Relational Algebra (youtube.com) Operators in Relational Model – YouTube		
Unit 3:	DBMS L4B Joins, Set Operations and Aggregate Functions (youtube.com) DBMS L5A Nested Subqueris (youtube.com) DBMS L6A Intermediate SQL (youtube.com) DBMS L7 Advanced SQL (youtube.com) DBMS L12A Indexing and Hashing (youtube.com)		
Unit 4	DBMS L15 Transactions – YouTube DBMS L16A Concurrency Control - YouTube DBMS L16B Concurrency Control (youtube.com) DBMS L16C Concurrency Control (youtube.com)		
Unit 5	DBMS L10A Application Design and Development - YouTube		

Syllabus

<u>DBMS L10B Application Design and Development (youtube.com)</u> <u>DBMS L19 Distributed Data Stores and NoSQL Databases (youtube.com)</u> <u>DBMS L18B Map Reduce and Hadoop - YouTube</u> <u>NoSQL Databases #1 (Data Models, CAP Theorem, BASE Property) - YouTube</u> https://youtu.be/ekuQjQUnj20?si=aL4T12EkHBZsvEK								
Mode of Evaluation								
CIE							ESE	Total
ST1	ST2	ST3	TA1	TA2	TA3	Attendance		
			5	5	5	5		
30			20				100	150

Branch wise Applications

- Railway Reservation System
- Library Management System
- Banking
- Education Sector
- Credit card exchanges
- Social Media Sites
- Broadcast communications
- Account
- Online Shopping
- Human Resource Management
- Manufacturing
- Airline Reservation System
- Healthcare

Prerequisite and Recap

- It is recommended to have fundamental computer knowledge that includes concepts of computer architecture,
- storage and hardware. Knowledge of data structures and algorithms and programming will be an added benefit.

Recap

In last unit we studied about

- Operators and predicates in Database
- MySQL nested queries
- Database connectivity with JAVA/Python

Unit Content

Transaction & concurrency control

- Transaction system: - Life cycle of transaction, ACID Properties
- Schedule & Types of Schedule, Serializability, Recoverability
- Concurrency Control Techniques: Concurrency Control, Locking
- Techniques for concurrency control, 2-phase Locking protocol
- Transaction & Data Control: -Grant, Revoke, commit & Rollback
- Deadlock Handling, Timestamping, Validation Based Protocol

Topic Objectives

1. The DBMS is used to schedule the access of data concurrently.
2. It means that the user can access multiple data from the database without being interfered with each other.
3. Transactions are used to manage concurrency.
4. It is also used to satisfy ACID properties.

Transaction

Goal :- Understand the basic properties of a transaction and learn the concepts underlying transaction processing as well as the concurrent executions of transactions.

A transaction is a unit of a program execution that accesses and possibly modifies various data objects (tuples, relations). **Or**

- Collections of operations that form a single logical unit of work are called transactions.
- A database system must ensure proper execution of transactions despite failures — either the entire transaction executes, or none of it does.

Transaction Operations are :- Read (),Write ();

Example

E.g., transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Two main issues to deal with:

- Failures of various kinds, such as hardware failures and system crashes
- Concurrent execution of multiple transactions

Properties of Transaction

To preserve the integrity of data the database system must ensure ACID Properties.

ACID Properties are :-

1. Atomicity
2. Durability
3. Consistency
4. Isolation

ACID Properties

1. **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
2. **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
3. **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - ❖ That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
4. **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

Example of Fund Transfer

- Transaction to transfer \$50 from account A to account B:
 1. **read**(A)
 2. $A := A - 50$
 3. **write**(A)
 4. **read**(B)
 5. $B := B + 50$
 6. **write**(B)
- **Atomicity requirement**
 - if the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
 - Failure could be due to software or hardware
 - the system should ensure that updates of a partially executed transaction are not reflected in the database
- **Durability requirement** — once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.

Continued

- Transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)

Consistency requirement in above example:

- the sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
 - Explicitly specified integrity constraints such as primary keys and foreign keys
 - Implicit integrity constraints
 - e.g. sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction must see a consistent database.
 - During transaction execution the database may be temporarily inconsistent.
 - When the transaction completes successfully the database must be consistent
 - Erroneous transaction logic can lead to inconsistency

Continued

- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).

T1

1. **read**(A)
2. $A := A - 50$
3. **write**(A)

T2

4. **read**(B)
5. $B := B + 50$
6. **write**(B)

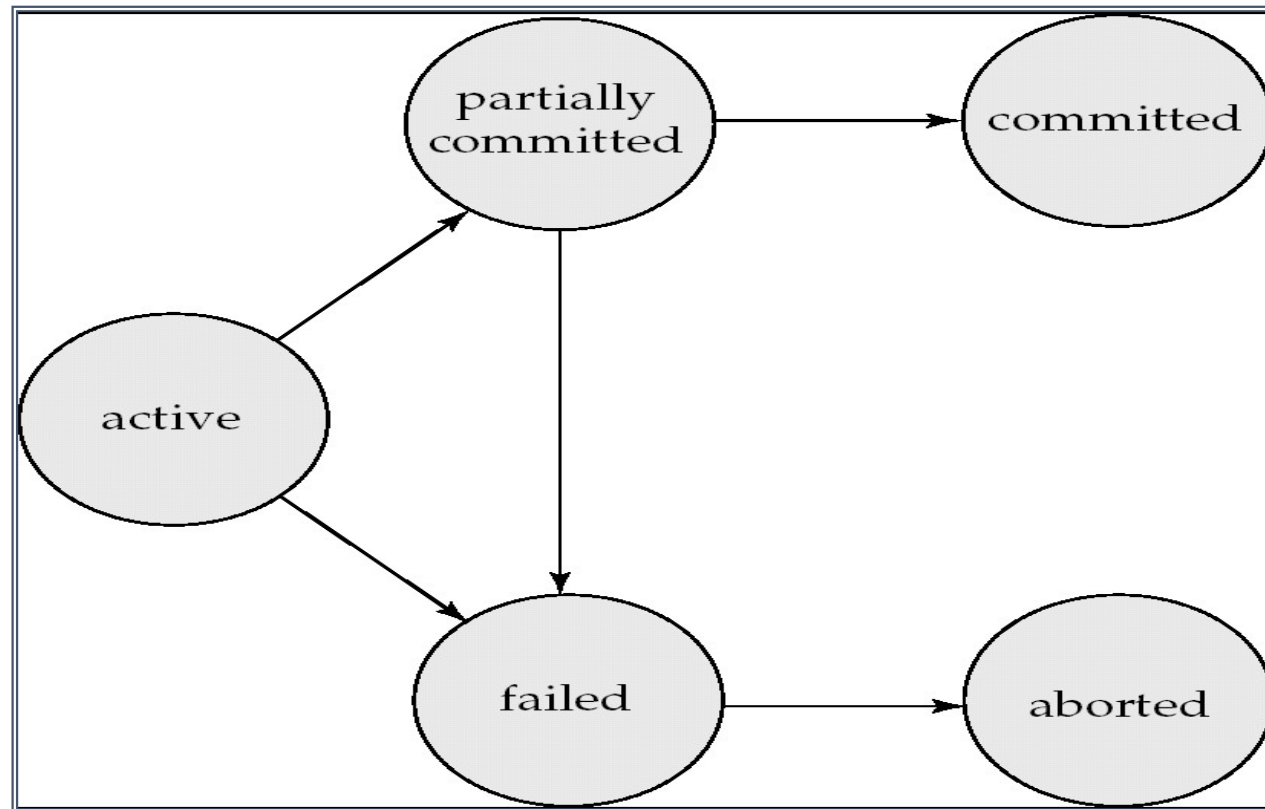
read(A), read(B), print($A+B$)

- Isolation can be ensured trivially by running transactions **serially**
 - that is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.

Transaction States(CO4)

- ❖ **Active** – the initial state; the transaction stays in this state while it is executing
- ❖ **Partially committed** – after the final statement has been executed.
- ❖ **Failed** -- after the discovery that normal execution can no longer proceed.
- ❖ **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - Restart the transaction
 - can be done only if no internal logical error
 - Kill the transaction
- ❖ **Committed** – after successful completion.

Transaction State Diagram



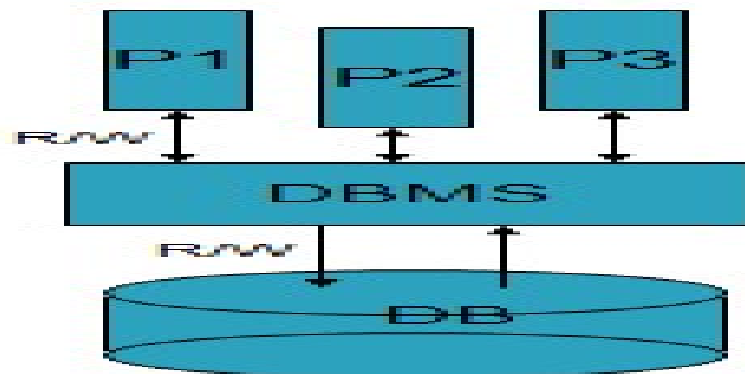
Concurrent Execution

Multiple transactions are allowed to run concurrently in the system. Advantages are:-

- **Increased processor and disk utilization**, leading to better transaction *throughput*
 - E.g. one transaction can be using the CPU while another is reading from or writing to the disk.
- **Reduced average response time** for transactions: short transactions need not wait behind long ones.

Challenges:-

- Concurrency Control: How do the DBMSs handle concurrent transactions?
- Crash Recovery: How do the DBMSs handle partial transactions because of machine crashes or users abort the transactions ?



Short Quiz

1. A transaction is delimited by statements (or function calls) of the form

- a) **Begin transaction and end transaction**
- b) Start transaction and stop transaction
- c) Get transaction and post transaction
- d) Read transaction and write transaction

2. Identify the characteristics of transactions

- a) Atomicity
- b) Durability
- c) Isolation
- d) **All of the mentioned**

Schedule

We know that **transactions** are set of instructions and these instructions perform operations on database.

When multiple transactions are running concurrently then there needs to be a sequence in which the operations are performed because at a time only one operation can be performed on the database. This sequence of operations is known as **Schedule**.

Schedule – A sequence of instructions that specify the chronological order in which instructions of concurrent transactions are executed.

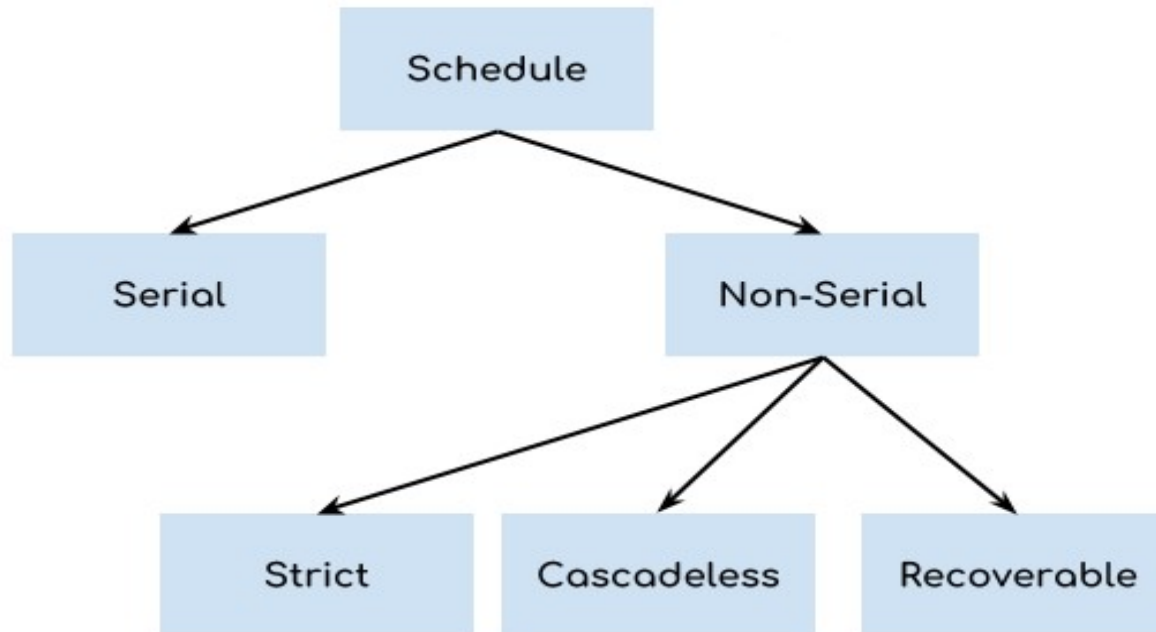
Continued

A schedule is a collection of many transactions which is implemented as a unit.

Depending upon how these transactions are arranged in within a schedule, a schedule can be of two types:-

- 1.Serial: The transactions are executed one after another, in a non-preemptive manner.**
- 2.Concurrent: The transactions are executed in a preemptive, time shared method.**

Types of Schedule



Continued

1. Serial Schedule:- In **Serial schedule**, a transaction is executed completely before starting the execution of another transaction. In other words, you can say that in serial schedule, a transaction does not start execution until the currently running transaction finished execution. This type of execution of transaction is also known as **non-interleaved** execution.

2. Non Serial Schedule :- In **Non Serial schedule**, a transaction is executed completely in context switching mode.(Concurrent Execution).

Note:- The transactions are executed in a non-serial manner, keeping the end result correct and same as the serial schedule.

Schedule 1: Serial Schedule

- Let T_1 transfer 50 from A to B , and T_2 transfer 10% of the balance from A to B .
- A serial schedule in which T_1 is followed by T_2 :-

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code>

- In this example, the transaction T_2 does not start execution until the transaction T_1 is finished.

Serial Schedule

Schedule 1: serial schedule :-

T2 is a new transaction which deposits to account B 10% of the amount in account A. If we prepare a serial schedule, then either T1 will completely finish before T2 can begin, or T2 will completely finish before T1 can begin.

Schedule 2: Serial Schedule

A serial schedule where T_2 is followed by T_1

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$

Schedule 3: Non-Serial Schedule

- ▶ However, if we want to create a concurrent schedule, then some Context Switching need to be made, so that some portion of T_1 will be executed, then some portion of T_2 will be executed and so on.
- ▶ Let T_1 and T_2 be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1.

Schedule 3: Non-Serial Schedule

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$
$\text{read}(B)$ $B := B + 50$ $\text{write}(B)$	$\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$

In Schedules 1, 2 and 3, the sum $A + B$ is preserved.

Schedule 4: Non-Serial Schedule

The following concurrent schedule does not preserve the value of $(A + B)$.

T_1	T_2
<code>read(A)</code> <code>A := A - 50</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + 50</code> <code>write(B)</code>	<code>read(A)</code> <code>temp := A * 0.1</code> <code>A := A - temp</code> <code>write(A)</code> <code>read(B)</code> <code>B := B + temp</code> <code>write(B)</code>

Prerequisite and Recap

Prerequisite:

- Concept of transactions
- Basic understanding of multiple transactions execution

Recap:

- Types of schedules (serial, non-serial)
- Serializability and recoverability concepts

Topic Objective

Schedule, Serializability, Recoverability

- To learn different types of schedules in DBMS.
- To understand serializability for correct transaction execution.
- To analyze recoverability to maintain database consistency after failures.

Serializability

- When multiple transactions are running concurrently then there is a possibility that the database may be left in an inconsistent state.

Serializability is a concept that helps us to check which **schedules** are serializable. A serializable schedule is the one that always leaves the database in consistent state.

- A **serial schedule** is always a serializable schedule because in serial schedule, a transaction only starts when the other transaction finished execution.
- However a non-serial schedule needs to be checked for Serializability.

Continued

A **non-serial schedule** of n number of transactions is said to be serializable schedule, if it is equivalent to the serial schedule of those n transactions.

A **serial schedule** doesn't allow concurrency, only one transaction executes at a time and the other starts when the already running transaction finished.

- ▶ **Basic Assumption** – Each transaction preserves database consistency.
- ▶ Thus serial execution of a set of transactions preserves database consistency.

Note :- A schedule is serializable if it is equivalent to a serial schedule.

There are Two type of Serializability :-

1. **conflict serializability:** A non serial schedule is converted into serial schedule by swapping non conflicting operations then it is known as conflict serializable schedule.
2. **view serializability:-** If a schedule is view equivalent to its serial schedule then the given schedule is said to be View Serializable.

Conflict Serializability (CO4)

- A schedule is called conflict serializable if after swapping of non-conflicting operations, it can transform into a serial schedule.
- The schedule will be a conflict serializable if it is conflict equivalent to a serial schedule.
- Two instructions of two different transactions may want to access the same data item in order to perform a read/write operation. Conflict Serializability deals with detecting whether the instructions are conflicting in any way, and specifying the order in which these two instructions will be executed in case there is any conflict.

A conflict arises if at least one (or both) of the instructions is a write operation.

- i. $T1 R(x) = T2 R(x)$ // non Conflict
- ii. $T1 R(x) = T2 W(x)$ // Conflict
- iii. $T1 W(x) = T1 R(x)$ // Conflict
- iv. $T1 W(x) = T2 W(x)$ // Conflict

Rules for Conflict Serializability (CO4)

The following rules are important in Conflict Serializability:

1. If two instructions of the two concurrent transactions are both for read operation, then they are not in conflict, and can be allowed to take place in any order.
2. If one of the instructions wants to perform a read operation and the other instruction wants to perform a write operation, then they are in conflict, hence their ordering is important. If the read instruction is performed first, then it reads the old value of the data item and after the reading is over, the new value of the data item is written. If the write instruction is performed first, then updates the data item with the new value and the read instruction reads the newly updated value.

Conflict Serializability (CO4)

3. If both the transactions are for write operation, then they are in conflict but can be allowed to take place in any order, because the transaction do not read the value updated by each other. However, the value that persists in the data item after the schedule is over is the one written by the instruction that performed the last write.

Finally we can say:-

Two instruction/operation are said to be in conflict Operation, if they satisfy all the following three conditions:-

1. Two operation are set of conflicting when they should belong to different transactions.
2. Both the instruction are working on same data item.
3. At least one of the instruction is a write instruction.

Conflict Equivalent Schedules (CO4)

- Two schedules are said to be conflict Equivalent if one schedule can be converted into other schedule after swapping non-conflicting operations.

Two schedules are said to be conflict equivalent if and only if:

- They contain the same set of the transaction.
- If each pair of conflict operations are ordered in the same way.

Continued

Methods for conflict Serializability are :-

1. **Swapping:-** A schedule is called conflict serializable if we can convert it into a serial schedule after swapping its non-conflicting operations.
2. **Precedence Graph:-** A precedence graph is a directed acyclic graph used in computer science to represent dependencies between tasks, processes, or database transactions. It defines execution order: a node pointing to means must finish before starts. In databases, it detects conflict serializability, where a cycle indicates a non-serializable schedule.

Conflicting and Non-conflicting operations(CO4)

Instructions l_i and l_j of transactions T_i and T_j respectively, **conflict** if and only if there exists some item Q accessed by both l_i and l_j , and at least one of these instructions wrote Q .

1. $l_i = \text{read}(Q)$, $l_j = \text{read}(Q)$. l_i and l_j don't conflict.
 2. $l_i = \text{read}(Q)$, $l_j = \text{write}(Q)$. They conflict.
 3. $l_i = \text{write}(Q)$, $l_j = \text{read}(Q)$. They conflict
 4. $l_i = \text{write}(Q)$, $l_j = \text{write}(Q)$. They conflict
- It seem that , a conflict between l_i and l_j forces a (logical) temporal order between them.
 - If l_i and l_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule.

Conflict Serializability by swapping(CO4)

Example :-

- Schedule S1 can be transformed into Schedule S2, a serial schedule where T_2 follows T_1 , by series of swaps of non-conflicting instructions.
- Therefore Schedule S1 is conflict serializable.

Non-serial schedule

T1	T2
Read(A) Write(A)	
	Read(A) Write(A)
Read(B) Write(B)	
	Read(B) Write(B)

Schedule S1

Serial Schedule

T1	T2
Read(A) Write(A) Read(B) Write(B)	
	Read(A) Write(A) Read(B) Write(B)

Schedule S2

Continued

In Non Serial Schedule S1

1. Two instruction are set of conflicting when they should belong to different transactions. YES
2. Both the operations are working on same data item. NO
(Such case instruction are not conflict swap it)
- 3 . At least one of the operation is a write instruction.
(Don't need to check because 2nd condition is disqualified for conflict operation)

So Non Serial schedule (schedule S1) is Conflict Serializable and also a serial schedule if this is a serial schedule maintain the consistency of database.

Precedence Graph(CO4)

A **precedence graph**, also known as **serialization graph** or **conflict graph**, is used for testing Conflict Serializability of a schedule in the condition that forms the setting of concurrency control in databases.

In a schedule of a **precedence graph**, The total number of transactions will be similar to the total number of nodes.

Nodes: In the graph, for each transaction **T_p** the graph contains a single node.

Edges: An edge is regulated connectivity between a set of two distinct transactions T_q and T_r and it shows in the format T_q → T_r, where T_q is the beginning of the edge and T_r is the ending.

View Serializabilty

- ❖ This is another type of serializability that can be derived by creating another schedule out of an existing schedule, involving the same set of transactions.
- ❖ It is a type of serializability that can be used to check whether the given schedule is view serializable or not.
- ❖ A schedule called as a view serializable if it is view equivalent to a serial schedule.

Need of View Serializability

We know that a serial schedule never leaves the database in inconsistent state because there are no concurrent transactions execution.

However a non-serial schedule can leave the database in inconsistent state because there are multiple transactions running concurrently. By checking that a given non-serial schedule is view serializable, we make sure that it is a consistent schedule.

You may be wondering instead of checking that a non-serial schedule is serializable or not, can't we have serial schedule all the time?

The answer is no, because concurrent execution of transactions fully utilize the system resources and are considerably faster compared to serial schedules.

View Equivalent

Example:-

T1	T2
-----	-----
R(X)	
W(X)	
	R(X)
	W(X)
R(Y)	
W(Y)	
	R(Y)
	W(Y)

Serial Schedule of the above given schedule:

T1	T2
-----	-----
R(X)	
W(X)	
R(Y)	
W(Y)	
	R(X)
	W(X)
	R(Y)
	W(Y)

As we know that in [Serial schedule](#) a transaction only starts when the current running transaction is finished.

If we can prove that the given schedule is **View Equivalent** to its serial schedule then the given schedule is called **view Serializable**.

Condition of Schedules to be View Equivalent

Condition of schedules to be View-equivalent

- Consider two schedules S1 and S2 each consisting of two transactions T1 and T2.
- Schedules S1 and S2 are called view equivalent if the following three conditions hold true for them-

Note:- Let us consider that the transactions T1 and T2 are being serialized to create two different schedules S1 and S2 which we want to be **View Equivalent** and **both T1 and T2 wants to access the same data item**.

1)Initial Read

For each data item A, if transaction T_i reads A from the database initially in schedule S1, then in schedule S2 also, T_i must perform the initial read of A from the database. ("Initial readers must be same for all the data items".)

Transaction T1 is reading A from database.

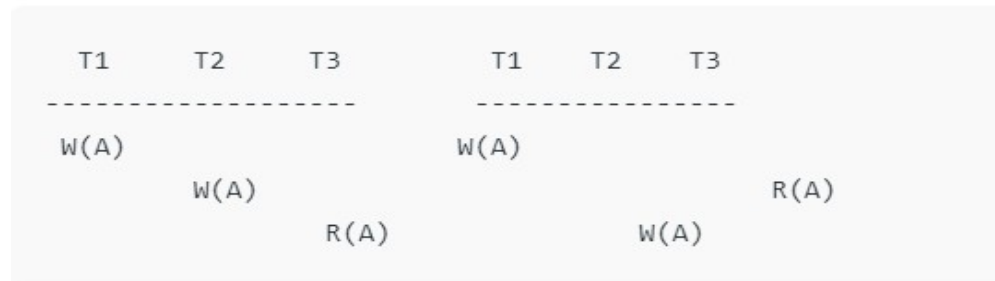
T1	T2	T1	T2
R(A)		R(A)	
W(A)	W(A)	W(A)	W(A)

Continued

2) Updated Read or intermediate read

If transaction T_i reads a data item that has been updated by the transaction T_j in schedule S_1 , then in schedule S_2 also, transaction T_i must read the same data item that has been updated by the transaction T_j .

Write-read sequence must be same.



Above two schedule are not view-equivalent as in S_1 :T3 is reading A updated by T2, in S_2 T3 is reading A updated by T1.

Continued

3) Final Write operation

For each data item A , if A has been updated at last by transaction T_i in schedule S_1 , then in schedule S_2 also, A must be updated at last by transaction T_i .

“Final writers must be same for all the data items”.

T1	T2	T1	T2
-----		-----	
R(A)		R(A)	
	W(A)	W(A)	
W(A)			W(A)

Above two schedules are not view-equivalent as Final write operation in S_1 is done by T_1 while in S_2 done by T_2 .

Example

Lets take an example.

Non-Serial

S1	
T1	T2
R(X)	
W(X)	
	R(X)
	W(X)
R(Y)	
W(Y)	
	R(Y)
	W(Y)

Serial

S2	
T1	T2
R(X)	
W(X)	
R(Y)	
W(Y)	
	R(X)
	W(X)
	R(Y)
	W(Y)

S2 is the serial schedule of S1. If we can prove that they are view equivalent then we can say that given schedule S1 is view Serializable

Lets check the three conditions of view serializability.

1. Initial Read

- In schedule S1, transaction T1 first reads the data item X. In S2 also transaction T1 first reads the data item X.
- Lets check for Y. In schedule S1, transaction T1 first reads the data item Y. In S2 also the first read operation on Y is performed by T1.
- We checked for both data items X & Y and the **initial read** condition is satisfied in S1 & S2.

Continued

2. Update Read

- In S1, transaction T2 reads the value of X, written by T1. In S2, the same transaction T2 reads the X after it is written by T1.
- In S1, transaction T2 reads the value of Y, written by T1. In S2, the same transaction T2 reads the value of Y after it is updated by T1.
- The update read condition is also satisfied for both the schedules.

3. Final Write

- In schedule S1, the final write operation on X is done by transaction T2. In S2 also transaction T2 performs the final write on X.
- Lets check for Y. In schedule S1, the final write operation on Y is done by transaction T2. In schedule S2, final write on Y is done by T2.
- We checked for both data items X & Y and the **final write** condition is satisfied in S1 & S2.

Continued

Result:-

Since all the three conditions that checks whether the two schedules are view equivalent are satisfied in this example, which means S1 and S2 are view equivalent.

Also, as we know that the schedule S2 is the serial schedule of S1, thus we can say that the schedule S1 is view serializable schedule.

Checking for a schedule to be view serializable(CO4)

Checking Whether a Schedule is View Serializable Or Not-

Method-01:-

Check whether the given schedule is conflict serializable or not.

- ❖ If the given schedule is conflict serializable, then it is surely view serializable. Stop and report your answer.
- ❖ If the given schedule is not conflict serializable, then it may or may not be view serializable. Go and check using other methods.

Note:-

1. All conflict serializable schedules are view serializable.
2. All view serializable schedules may or may not be conflict serializable.

Continued

Method-02:

Check if there exists any blind write operation. (Writing without reading is called as a blind write).

- ❖ If there does not exist any blind write, then the schedule is surely not view serializable. Stop and report your answer.
- ❖ If there exists any blind write, then the schedule may or may not be view serializable. Go and check using other methods.

Note:-

No blind write means not a view serializable schedule.

Continued

Method-03:

In this method, try finding a view equivalent serial schedule.

- Then, draw a graph using those dependencies.
- If there cycle exists in the graph, then the schedule is view serializable or not check it.
- By using the above three conditions, write all the dependencies.

1. What is the primary advantage of a "Non-serial Schedule"?
 - A) It allows transactions to execute in parallel, improving system performance**
 - B) It avoids errors and ensures all transactions are isolated
 - C) It ensures that all transactions are executed without conflicts
 - D) It guarantees that no transaction can affect the system

2. What is the significance of "Transaction Isolation" in a transaction system?
 - A) It ensures that transactions are executed serially without any interleaving
 - B) It guarantees that a transaction's operations are not visible to other transactions until they are committed**
 - C) It ensures that no transaction can be rolled back once it starts
 - D) It prevents errors during transaction validation

3. What is the main purpose of the "ACID" properties in transaction management?
 - A) To ensure that transactions are executed quickly and without errors
 - B) To guarantee that database operations are safe, reliable, and maintain data integrity**
 - C) To allow multiple transactions to be executed simultaneously
 - D) To optimize database performance

Short Quiz

1. I and J are _____ if they are operations by different transactions on the same data item, and at least one of them is a write operation.
 - a) **Conflicting**
 - b) Overwriting
 - c) Isolated
 - d) Durable

2. A schedule is _____ if it is conflict equivalent to a serial schedule.
 - a) **Conflict serializable**
 - b) Conflicting
 - c) Non serializable
 - d) None of the mentioned

Recoverability

Basic :-

- The order of execution of the transaction is called a schedule, may it be a serial schedule or a concurrent schedule.
- Let's say, there are 3 transactions running simultaneously each having n , m , and k sub-operations. Then all the possible serial schedules are $3! = 6$. The number of concurrent schedules possible = Total _ serial.

Continued

Now due to this concurrency in schedules there can arise a lot of issues such as inconsistent schedule.

Now if this inconsistency happens or there is a any crash in the system then our schedule should be able to revert back or rollback to the original state. **Thus the concept of recoverability comes into the picture.**

What is a recoverable schedule?

- A Schedule in which transactions commit only after all transactions whose changes they read , then commit are called **recoverable schedules**.
- In other words, if some transaction T_j is reading value updated or written by some other transaction T_i , then the commit of T_j must occur after the commit of T_i .

What is Irrecoverable schedule?

Irrecoverable schedule: The schedule will be irrecoverable if T_j reads the updated value of T_i and T_j committed before T_i commit.

Example

Let us understand this with a simple example: Let's say you are copying an examination paper from the person sitting in front of you. Then you should not submit/commit your paper until and unless the person in front of you submits first. Because he may later add some more points or change some answers which you will miss out.

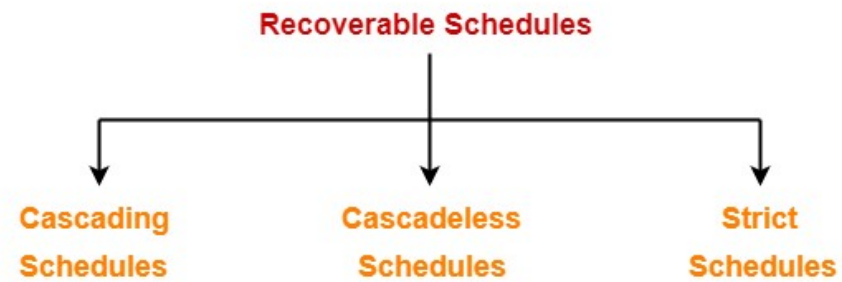
Example

S1 :		S2 :	
T ₁	T ₂	T ₁	T ₂
R(A)		R(A)	
	R(A)		W(A)
W(A)			W(B)
W(B)		R(B)	
	R(A)	W(B)	
	commit		commit
commit		commit	
Irrecoverable Schedule		Recoverable Schedule	

In the above figure, S1 is not recoverable because T2 is committing before T1 although it is reading from T1. S2 is a recoverable schedule because T1 is committing after T2 as it has read from T2.

Types of Recoverable Schedule

A recoverable schedule may be any one of these kinds-



Cascading Schedule

Cascading Schedule-

- If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.
- It simply leads to the wastage of CPU time.

Example-

T1	T2	T3	T4
R (A) W (A)			
	R (A) W (A)		
		R (A) W (A)	
			R (A) W (A)
Failure			

Cascading Recoverable Schedule

Continued

Here,

Transaction T2 depends on transaction T1.

Transaction T3 depends on transaction T2.

Transaction T4 depends on transaction T3.

In this schedule,

The failure of transaction T1 causes the transaction T2 to rollback.

The rollback of transaction T2 causes the transaction T3 to rollback.

The rollback of transaction T3 causes the transaction T4 to rollback.

Such a rollback is called as a **Cascading Rollback**.

There arises a problem of cascading rollbacks, which is a rollback of a single transaction that will lead to a rollback of many other transactions, which will lead to less throughput and high time-consuming. Now comes the concept of cascadeless schedule/avoiding cascading abort.

NOTE-

- If the transactions T2, T3 and T4 would have committed before the failure of transaction T1, then the schedule would have been irrecoverable.

Cascadeless Schedule

If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Cascadeless Schedule**.

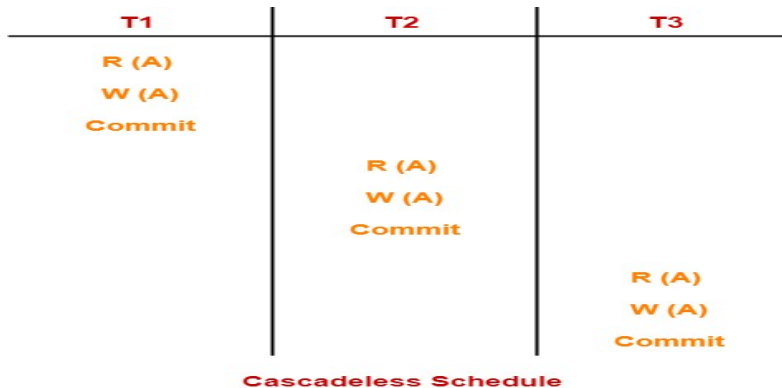
In other words,

- Cascadeless schedule allows only committed read operations.
- Therefore, it avoids cascading roll back and thus saves CPU time.

Example

Example

Here no uncommitted reads are allowed. That is no dirty reads are allowed. That is no W-R problem. Disallowing a transaction from reading uncommitted changes from another transaction. Cascadeless schedules are also recoverable.



In the above figure, the schedule is cascadeless because we are reading only after a committed write. But in a cascadeless schedule, there still can be a problem of W-W, or lost update problem.

So, to avoid that strict schedule comes into the picture.

Continued

NOTE-

- Cascadeless schedule allows only committed read operations.
- However, it allows uncommitted write operations.

Example-

T1	T2
R (A)	
W (A)	
	W (A) // Uncommitted Write
Commit	

Cascadeless Schedule

Strict Schedule

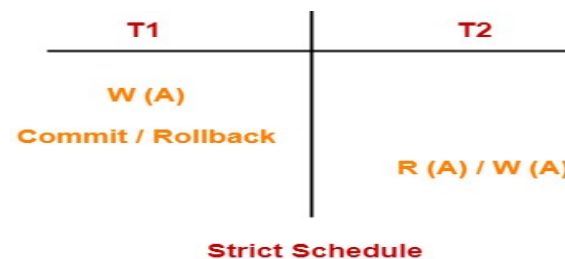
If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Strict Schedule**.

In a strict schedule if T1 performs W(A) then T2 is not allowed to R(A) or W(A) till T1 commits/rollback. Thus, avoiding the W-W(lost-update) problem and W-R problem. Strict schedules are cascadeless as well as recoverable.

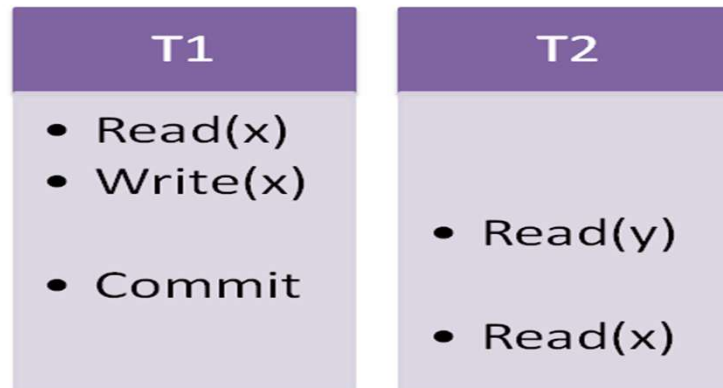
In other words,

- Strict schedule allows only committed read and write operations.
- Clearly, strict schedule implements more restrictions than cascadeless schedule.

Example :-



Example



In the above figure, this schedule is strict because T2 is doing R(X) only after committing T1. R(Y) is before commit but that is okay because it is on different data.

Remember-

- Strict schedules are more strict than cascadeless schedules.
- All strict schedules are cascadeless schedules.
- All cascadeless schedules are not strict schedules.

Continued

In addition ,

serial schedule $\subseteq$ strict schedule $\subseteq$ cascadeless schedule $\subseteq$ recoverable schedule

Prerequisite and Recap

Prerequisite:

- Concurrency in DBMS
- Basic idea of transaction conflicts

Recap:

- Deadlock concept and handling methods
- Timestamp and validation-based protocols

Topic Objective

Deadlock, Timestamping, Validation Protocol

- To understand deadlock situations and handling techniques.
- To study timestamp-based concurrency control.
- To learn validation-based protocols for transaction management.

Deadlock Handling

What is Deadlock?

Deadlock is a situation where **two or more transactions are waiting for each other's resources**, and none can proceed.

Example:

- T1 holds Resource A, waiting for B
 - T2 holds Resource B, waiting for A
- Both are stuck → **Deadlock**

Conditions for Deadlock (Necessary Conditions)

Deadlock occurs if all these are true:

- **Mutual Exclusion** – Resource used by one at a time
- **Hold and Wait** – Holding one resource, waiting for another
- **No Preemption** – Resource cannot be taken forcefully
- **Circular Wait** – Transactions waiting in a cycle

Continued

Deadlock handling includes:

- **Prevention**
- **Detection and Recovery**

Deadlock Prevention

Deadlock Prevention:- This protocol ensure that the system does not goes to deadlock state

There are two approaches to deadlock prevention.

1. One approach ensures that no cyclic waits can occur by ordering the requests for locks.
2. Second Approach is the rollback of transaction. The simplest scheme under the first approach requires that each transaction locks all its data before execution. Moreover, either all are locked or none.

Another Approach for preventing deadlocks is to impose an ordering of all data items, and to require that transaction lock data items in sequence.

Continued

The second approach for prevention of deadlock is preemption and transaction rollback.

In preemption, when a transaction T2 requests a lock that transaction T1 holds, the lock granted to T1 may be preempted by rolling back of T1 and granting of lock to T2.

Two different deadlock prevention Technique using Timestamps

- 1.Wait-Die Scheme
- 2.Wound-Wait Scheme

Continued

Wait-Die scheme

- In this scheme, if a transaction requests for a resource which is already held with a conflicting lock by another transaction then the DBMS simply checks the timestamp of both transactions. It allows the older transaction to wait until the resource is available for execution.
- Let's assume there are two transactions T_i and T_j and let $TS(T)$ is a timestamp of any transaction T . If T_2 holds a lock by some other transaction and T_1 is requesting for resources held by T_2 then the following actions are performed by DBMS:-
- Check if $TS(T_i) < TS(T_j)$ - If T_i is the older transaction and T_j has held some resource, then T_i is allowed to wait until the data-item is available for execution. That means if the older transaction is waiting for a resource which is locked by the younger transaction, then the older transaction is allowed to wait for resource until it is available.

Continued

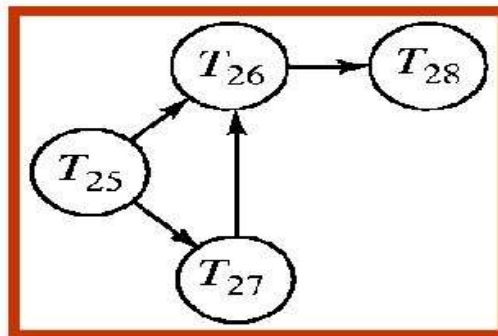
B. Wound wait scheme

- In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After the minute delay, the younger transaction is restarted but with the same timestamp.
- If the older transaction has held a resource which is requested by the younger transaction, then the younger transaction is asked to wait until older releases it.

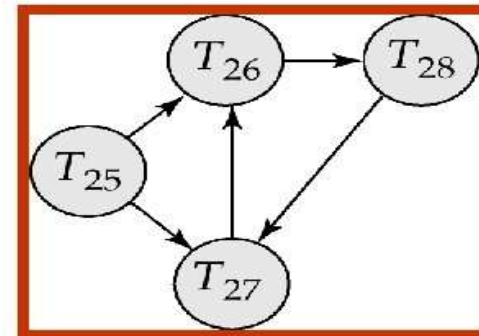
Deadlock Detection

Deadlocks can be described precisely in terms of a directed Graph called wait-for Graph.

Deadlock Detection (Cont.)



Wait-for graph without a cycle



Wait-for graph with a cycle

Deadlock Recovery

When a detection algorithm determines that a dead lock exist, the system must recover from the deadlock.

The most common solution is to rollback the transaction to break the deadlock. The following three actions should be taken: -

1.Selection Of Victim:- Given a set of deadlocked transactions, we should determine which transaction to rollback to break the deadlock. Following should be kept in mind for determining the roll back of transaction

- a)How many data items the transaction has used?
- b)How many more data items the transaction needs for it to complete?
- c)How many transactions will be involved in rollback?

Continued

b)Rollback :- Once we have decided that a particular transaction must be rolled back, we must determine how far this transaction should be rolled back. The simplest solution is total roll back. Abort the transaction and restart it.

However it is efficient to roll back the transaction to break the deadlock. The deadlock detection mechanism should decide which locks the selected transaction needs to release in order to break the deadlock.

c)Starvation:- In a system where the selection of victims is based primarily on cost factors. It may happen that the same transaction is always picked as a victim.

As a result, the transaction never completes its designated task, thus there is a transaction failure. We must ensure that a particular transaction is rolled back for finite times.

Failure Classification

1. Transaction failure : The transaction failure occurs when it fails to execute or when it reaches a point from where it can't go any further. If a few transaction or process is hurt, then this is called as transaction failure.

- Reasons for a transaction failure could be –
 - **Logical errors:** transaction cannot complete due to some internal error condition
 - **System errors:** the database system must terminate an active transaction due to an error condition (e.g., deadlock).

2. System crash: a power failure or other hardware or software failure causes the system to crash.

- **Fail-stop assumption:** non-volatile storage contents are assumed to not be corrupted by system crash
 - Database systems have numerous integrity checks to prevent corruption of disk data

Continued

3. Disk failure: a head crash or similar disk failure destroys all or part of disk storage

-Disk failure occurs due to the formation of bad sectors, disk head crash, and unreachability to the disk or any other failure, which destroy all or part of disk storage.

Concurrency Control

- A database must provide a mechanism that will ensure that all possible schedules are
 - either conflict or view serializable, and
 - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
 - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.

Continued

- **Concurrency Control**

It is process of managing simultaneous execution of transaction in a shared database to ensure the serializability of transaction.

Concurrency control in DBMS is a mechanism that ensures multiple transactions can execute simultaneously without causing inconsistency in the database. In a multi-user environment, several users may access and modify the same data at the same time, which can lead to problems such as lost updates, dirty reads, non-repeatable reads, and phantom reads. To avoid these issues, concurrency control maintains the isolation and consistency properties of transactions, which are part of the ACID properties. The main objective of concurrency control is to ensure that the outcome of concurrent transactions is equivalent to some serial execution, thereby maintaining data integrity and correctness.

- **Purpose of Concurrency Control**

- To enforce Isolation
- To preserve database consistency
- To resolve the read write or write – write conflicts.

Concurrency Control Techniques

Concurrency Control Techniques:-

Concurrency control techniques are used to manage simultaneous transaction execution in a database system. The three main techniques are locking, timestamp ordering, and validation-based protocols.

- Time Stamping Protocol
- Lock Based Protocol
 - 2PL (2Phase Locking)
 - Graph Based Approach
- Validation Based Protocol

Time-stamp based protocols

- The timestamp ordering protocol is another concurrency control technique that assigns a unique timestamp to each transaction when it starts. These timestamps determine the order in which transactions should be executed. Each data item in the database maintains two timestamps: a read timestamp, which indicates the last time the data was read, and a write timestamp, which indicates the last time the data was written.
- When a transaction performs a read or write operation, the system checks these timestamps to ensure that the operation does not violate the chronological order. If a conflict occurs, the transaction is rolled back and restarted with a new timestamp. This protocol ensures serializability without using locks, which eliminates the possibility of deadlocks. However, it may lead to frequent rollbacks, especially in systems with high contention, and can cause starvation of some transactions.

Time-stamp based protocols

- Each transaction is issued a timestamp when it enters the system. If an old transaction T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp $TS(T_j)$ such that $TS(T_i) < TS(T_j)$.
- The protocol manages concurrent execution such that the time-stamps determine the serializability order.
- In order to assure such behavior, the protocol maintains for each data item Q two timestamp values:
 - **W-timestamp**(Q) is the largest time-stamp of any transaction that executed **write**(Q) successfully.
 - **R-timestamp**(Q) is the largest time-stamp of any transaction that executed **read**(Q) successfully.

Continued

- The timestamp ordering protocol ensures that any conflicting **read** and **write** operations are executed in timestamp order.
- Suppose a transaction T_i issues a **read**(Q)
 1. If $TS(T_i) < \mathbf{W}\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) \geq \mathbf{W}\text{-timestamp}(Q)$, then the **read** operation is executed, and $\mathbf{R}\text{-timestamp}(Q)$ is set to $\mathbf{max}(\mathbf{R}\text{-timestamp}(Q), TS(T_i))$.

Continued

- Suppose that transaction T_i issues **write**(Q).
 1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced.
 - Hence, the **write** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q .
 - Hence, this **write** operation is rejected, and T_i is rolled back.
 3. Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

Continued

- A partial schedule for several data items for transactions with
- timestamps 1, 2, 3, 4, 5

T_1	T_2	T_3	T_4	T_5
				read (X)
read (Y)	read (Y)	write (Y) write (Z)		
	read (Z) abort			read (Z)
read (X)		write (W) abort	read (W)	
				write (Y) write (Z)

Locking Techniques

- Locking is one of the most widely used concurrency control techniques. In this approach, a transaction must obtain a lock on a data item before accessing it. There are two primary types of locks: shared locks and exclusive locks. A shared lock allows multiple transactions to read a data item simultaneously but does not allow any transaction to modify it. An exclusive lock, on the other hand, allows a transaction to both read and write a data item, and no other transaction can access that data item while the lock is held.
- To ensure proper execution, the Two-Phase Locking (2PL) protocol is used. In this protocol, a transaction first enters a growing phase, where it acquires all the locks it needs, and then enters a shrinking phase, where it releases the locks. Once a lock is released, no new locks can be acquired. Variants such as strict 2PL ensure that all locks are held until the transaction commits, improving consistency. However, locking techniques can lead to problems such as deadlocks, where two or more transactions wait indefinitely for each other, and starvation, where a transaction may never get access to required resources.

Continued

- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

- **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

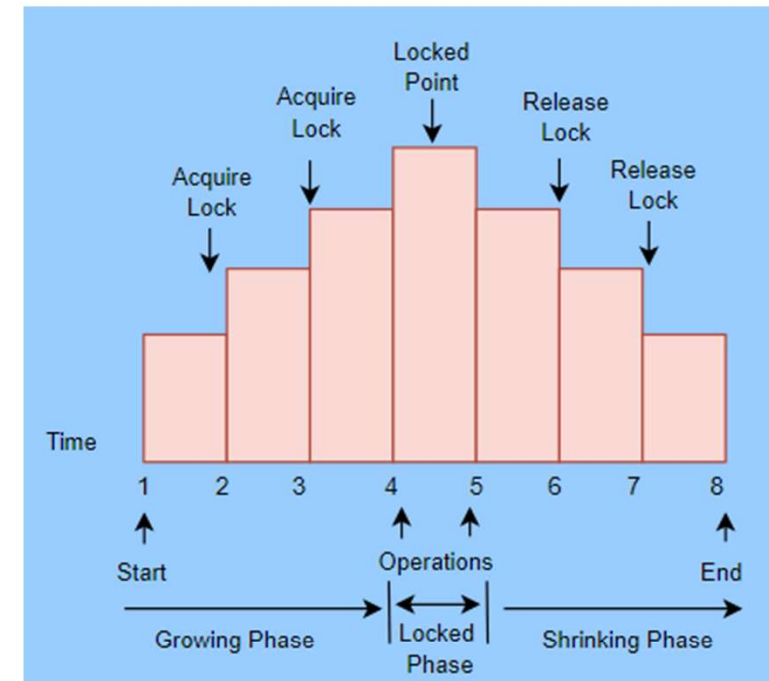
- A transaction may be granted a lock on a data item if the requested lock is compatible with locks already held on the data item by other transactions
- Any number of transactions can hold shared locks on a data item,
 - but if any transaction holds an exclusive lock on the data item no other transaction may hold any lock on that data item.
- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Example

- **Example** of a transaction performing locking:
 T_2 : **lock-S**(A);
 read (A);
 unlock(A);
 lock-S(B);
 read (B);
 unlock(B);
 display($A+B$)
- Locking as above is not sufficient to guarantee serializability
 - if A and B get updated in-between the read of A and B , the displayed sum would be wrong.

2PL(Two phase locking)

The Two-Phase Locking (2PL) protocol divides the execution of a transaction into two distinct phases: the growing phase and the shrinking phase. In the growing phase, the transaction acquires all the locks it needs without releasing any locks. Once it releases its first lock, it enters the shrinking phase, during which it can only release locks and cannot acquire new ones. This ensures serializability of transactions. In strict 2PL, all exclusive locks are held until the transaction commits, preventing cascading rollbacks and improving data consistency.



Lock based protocols

- This is a protocol which ensures conflict-serializable schedules.
- Phase 1: Growing Phase
 - transaction may obtain locks
 - transaction may not release locks
- Phase 2: Shrinking Phase
 - transaction may release locks
 - transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e. the point where a transaction acquired its final lock).

2PL

- Two-phase locking *does not* ensure freedom from deadlocks
- Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called **strict two-phase locking**. Here a transaction must hold all its exclusive locks till it commits/aborts.
- **Rigorous two-phase locking** is even stricter: here *all* locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.

Continued

- There can be conflict serializable schedules that cannot be obtained if two-phase locking is used.
- However, in the absence of extra information (e.g., ordering of access to data), two-phase locking is needed for conflict serializability in the following sense:

Given a transaction T_i that does not follow two-phase locking, we can find a transaction T_j that uses two-phase locking, and a schedule for T_i and T_j that is not conflict serializable.

Continued

- Two-phase locking with lock conversions:
 - First Phase:
 - can acquire a lock-S on item
 - can acquire a lock-X on item
 - can convert a lock-S to a lock-X (upgrade)
 - Second Phase:
 - can release a lock-S
 - can release a lock-X
 - can convert a lock-X to a lock-S (downgrade)
- This protocol assures serializability. But still relies on the programmer to insert the various locking instructions.

Example

- Consider the partial schedule

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .

- Such a situation is called a **deadlock**.
 - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

Continued

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.
- **Starvation** is also possible if concurrency control manager is badly designed. For example:
 - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
 - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

Continued

- **Problems with 2-PL,**
 - Cascading Aborts and Deadlocks.
- There are three categories:
 - Strict 2-PL
 - Rigorous 2-PL
 - Conservative 2-PL

Types of 2PL

Conservative 2-PL –

- **Static 2-PL**, this protocol requires the transaction to lock all the data items it access before the Transaction begins execution by pre-declaring its read-set and write-set.
 - If any of the pre-declared items needed cannot be locked, the transaction does not lock any of the items, instead it waits until all the items are available for locking.

Conservative 2-PL is *Deadlock free* and but it does not ensure Strict schedule

Continued

Strict-2PL

- The first phase of Strict-2PL is same as 2PL.
- After acquiring all the locks in the first phase, the transaction continues to execute normally. But in contrast to 2PL, Strict-2PL does not release a lock after using it.
- Strict-2PL holds all the locks until the commit point and releases all the locks at a time.
- Strict-2PL does not have cascading abort as 2PL does.
- Following Strict 2-PL ensures that our schedule is:
 - Recoverable
 - Cascadeless

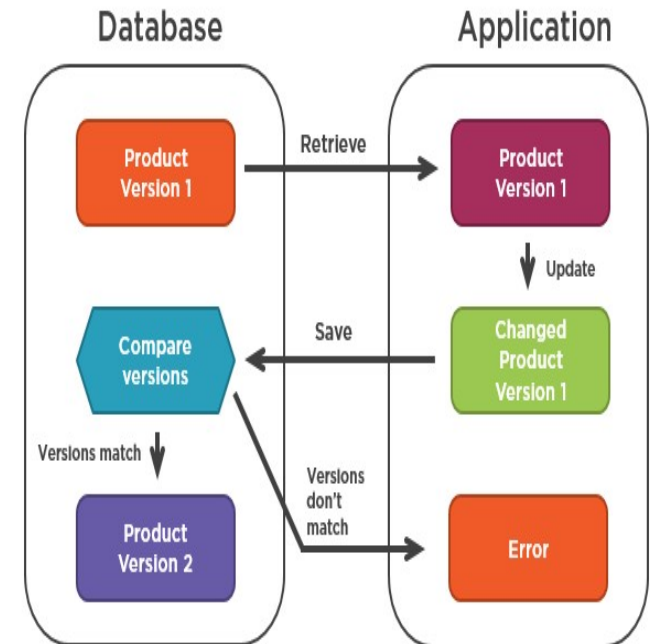
Continued

Rigorous 2-PL –

- This requires that in addition to the lock being 2-Phase **all Exclusive(X) and Shared(S) Locks** held by the transaction be released until *after* the Transaction Commits.
- Following Rigorous 2-PL ensures that our schedule is:
 - Recoverable
 - Cascadeless

Validation-Based Protocol

- The validation-based protocol, also known as optimistic concurrency control, divides transaction execution into three phases: read, validation, and write. In the read phase, the transaction reads data and performs operations on local copies without locking resources. In the validation phase, the system checks whether the transaction conflicts with other concurrent transactions.
- If the validation is successful, the transaction proceeds to the write phase, where changes are applied to the database. If validation fails, the transaction is rolled back and restarted. This approach improves concurrency but may lead to higher rollback rates in case of frequent conflicts.



Validation based protocol

Validation Based Protocol is also called Optimistic Concurrency Control Technique. This protocol is used in DBMS (Database Management System) for avoiding concurrency in transactions. It is called optimistic because of the assumption it makes, i.e. very less interference occurs, therefore, there is no need for checking while the transaction is executed.

- In this technique, no checking is done while the transaction is been executed.
- Until the transaction end is reached updates in the transaction are not applied directly to the database. All updates are applied to local copies of data items kept for the transaction.
- At the end of transaction execution, while execution of the transaction, a validation phase checks whether any of transaction updates violate serializability.
- If there is no violation of serializability the transaction is committed and the database is updated; or else, the transaction is updated and then restarted.

Continued

Optimistic Concurrency Control is a three-phase protocol. The three phases for validation based protocol:

Read Phase:

Values of committed data items from the database can be read by a transaction. Updates are only applied to local data versions.

Validation Phase:

Checking is performed to make sure that there is no violation of serializability when the transaction updates are applied to the database.

Write Phase:

On the success of the validation phase, the transaction updates are applied to the database, otherwise, the updates are discarded and the transaction is slowed down.

Thomas write rule

- Modified version of the timestamp-ordering protocol in which obsolete **write** operations may be ignored under certain circumstances.
- When T_i attempts to write data item Q , if $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of $\{Q\}$.
 - Rather than rolling back T_i as the timestamp ordering protocol would have done, this **{write}** operation can be ignored.
- Otherwise this protocol is the same as the timestamp ordering protocol.
- Thomas' Write Rule allows greater potential concurrency.
 - Allows some view-serializable schedules that are not conflict-serializable.

Lock based protocols

- A lock is a mechanism to control concurrent access to a data item
 - To access data item lock is acquired
 - After completion of transaction lock must be released
- Data items can be locked in two modes :
 1. *exclusive (X) mode*. Data item can be both read as well as write. X-lock is requested using **lock-X** instruction.
 2. *shared (S) mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to concurrency-control manager. Transaction can proceed only after request is granted.

1. What is a "lock" in the context of concurrency control?
 - A) A mechanism to restrict access to certain parts of a database to prevent conflicts**
 - B) A tool used to improve the performance of transaction queries
 - C) A method to allow multiple transactions to read data simultaneously
 - D) A temporary database table that stores transactional data

2. What is a "Shared Lock" in the context of concurrency control?
 - A) A lock that prevents all other transactions from accessing the data
 - B) A lock that allows multiple transactions to read the data but not modify it**
 - C) A lock that permits multiple transactions to write to the data
 - D) A lock that allows one transaction to write and others to read

3. Which of the following is a key feature of the 2-Phase Locking protocol?
 - A) It allows a transaction to release locks while still acquiring more locks
 - B) It ensures that a transaction can only release locks before acquiring them
 - C) It guarantees serializability by preventing cyclic dependencies between transactions**
 - D) It allows unrestricted transactions to execute concurrently

Prerequisites and Recap

Prerequisites

To understand Data Control Language (DCL), basic knowledge of DBMS, SQL commands (SELECT, INSERT, UPDATE, DELETE), and database structure is required. Understanding users, roles, and basic security concepts like authentication and authorization is also important.

Recap

Data Control Language (DCL) is used to control access to data in a database. It mainly includes GRANT and REVOKE commands, which allow or remove permissions for users. DCL helps ensure database security and restricts unauthorized access.

Topic Objectives

- The objective of DCL is to manage user permissions and ensure secure access to database resources.
- It helps protect data by allowing only authorized users to perform specific operations, maintaining data integrity and confidentiality.

Data Control Language

DCL (Data Control Language) includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system. These commands are used to control access to data in the database by granting or revoking permissions.

The primary DCL commands in SQL include:

GRANT: This command is used to give users access privileges to the database. These privileges can include the ability to select, insert, update, delete, and so on, over database objects like tables and views.

Syntax: GRANT privilege_name ON object_name TO user_name;

For example, **GRANT SELECT ON employees TO user123;**
gives user123 the permission to read data from the employees table.

REVOKE: This command is used to remove previously granted access privileges from a user.

Syntax: REVOKE privilege_name ON object_name FROM user_name;

For example, **REVOKE SELECT ON employees FROM user123;**
would remove user123's permission to read data from the employees table.

SQL COMMANDS:

GRANT:-

GRANT SELECT, UPDATE ON Customers TO user1;

REVOKE:-

REVOKE SELECT ON Customers FROM user1;

Data Control Language

- Database administrators typically use DCL commands. When using these commands, it's important to carefully manage who has access to what data, especially in environments where data sensitivity and user roles vary significantly.
- In some systems, DCL functionality also encompasses commands like DENY (specific to certain database systems like Microsoft SQL Server), which explicitly denies specific permissions to a user, even if those permissions are granted through another role or user group.
- Remember, the application and syntax of DCL commands can vary slightly between different SQL database systems, so it's always good to refer to specific documentation for the database you are using.

Transaction Control Language

- Transactions are atomic i.e. either every statement succeeds or none of statement succeeds. There are number of Transaction Control statements available that allow us to control this behavior. These statements ensure data consistency. TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.
- These operations are automatically committed in the database that's why they cannot be used while creating tables or dropping them.

Transaction Control Language

The primary TCL commands in SQL include:

COMMIT: This command is used to permanently save all changes made in the current transaction.

Syntax: COMMIT;

When you issue a COMMIT command, the database system will ensure that all changes made during the current transaction are saved to the database.

ROLLBACK: This command is used to undo changes that have been made in the current transaction.

Syntax: ROLLBACK;

If you issue a ROLLBACK command, all changes made in the current transaction are discarded, and the state of the data reverts to what it was at the beginning of the transaction.

SAVEPOINT: This command creates points within a transaction to which you can later roll back. It allows for partial rollbacks and more complex transaction control.

Syntax: SAVEPOINT savepoint_name;

You can roll back to a savepoint using ROLLBACK TO savepoint_name;

Transaction Control Language

The primary TCL commands in SQL include:

GRANT

The GRANT command is used to provide specific access privileges to users on database objects such as tables, views, or procedures. It allows database administrators to control who can perform operations like SELECT, INSERT, UPDATE, or DELETE, thereby ensuring security and controlled access.

REVOKE

The REVOKE command is used to remove previously granted privileges from users. It helps in restricting access when permissions are no longer required or when security policies change. This ensures that only authorized users have access to specific database operations.

Transaction Control Language

SQL COMMANDS:

COMMIT:-

```
UPDATE Customers  
SET country = 'UK'  
WHERE customer_id = 4;  
COMMIT;
```

ROLLBACK:-

```
DELETE FROM Orders;  
ROLLBACK;
```

SAVEPOINT:-

```
SAVEPOINT savepoint_name;
```

GRANT

The GRANT command is used to give permissions to users on database objects.

Syntax: GRANT privilege_name ON table_name TO user_name;

Example: GRANT SELECT, INSERT ON Employee TO user1;

REVOKE

The REVOKE command is used to remove permissions from users.

Syntax: REVOKE privilege_name ON table_name FROM user_name;

Example: REVOKE INSERT ON Employee FROM user1;

1. Which of the following is **not** a valid state in the life cycle of a transaction?

- a) Active
- b) Partially Committed
- c) Terminated**
- d) Failed

2. Data Manipulation Language (DML) is not to

- a) Create information table in the Database**
- b) Insertion of new information into the Database
- c) Deletion of information in the Database
- d) Modification of information in the Database

3. Which type of schedule does 2-phase locking protocol ensure?

- a) **Serializable**
- b) Recoverable
- c) Cascading
- d) Strict

4. In a strictly recoverable schedule, which condition is true?

- a) A transaction can commit before another transaction reads its data.
- b) Transactions must commit or roll back before reading the data of others.
- c) **It is guaranteed that a transaction that reads uncommitted data will not commit.**
- d) A transaction may not be rolled back once it commits.

5. Which statement is true regarding procedures?

- a) **They include procedural and SQL statements.**
- b) They work similarly to the functions.
- c) It does not need unique names.
- d) It cannot be created with SQL statements.

6. In 2PL, which of the following is true regarding lock acquisition?

- a) A transaction can release a lock during the growing phase.
- b) A transaction can acquire new locks during the shrinking phase.
- c) **A transaction must acquire all locks before starting the shrinking phase.**
- d) A transaction releases all locks during the growing phase.

7. What is the primary benefit of using SAVEPOINT in transaction management?
- a) It allows a transaction to be split into multiple smaller transactions.
 - b) It enables rolling back to a specific point without aborting the entire transaction.**
 - c) It ensures the transaction is committed.
 - d) It guarantees that a transaction will not fail.
8. In 2PL, which of the following is true regarding lock acquisition?
- a) A transaction can release a lock during the growing phase.
 - b) A transaction can acquire new locks during the shrinking phase.
 - c) A transaction must acquire all locks before starting the shrinking phase.**
 - d) A transaction releases all locks during the growing phase.

9. Which type of index is created automatically by the database when a **primary key** is defined?

- a) Unique index
- b) Clustered index**
- c) Non-clustered index
- d) Composite index

10. Which of the following statements is true about a **view** in SQL?

- a) Views cannot be updated.
- b) A view always stores data physically.
- c) Views are created to improve query performance.
- d) A view is based on a SELECT statement and does not store data.**

Weekly Assignment

- How does clustered indexing differ from non-clustered indexing?
- How can you implement time zone conversion between two different time zones in SQL?
- How do the ACID properties ensure data integrity in a DBMS?
- How does DBMS ensure recoverability in non-serial schedules?
- What is serializability? How do conflict serializability and view serializability differ?
- What is strict 2PL and how does it ensure recoverability in a database system?

Glossary Questions

- What are locking techniques used for concurrency control in a database, and why are they important?
- What is the 2-phase locking protocol, and how does it help maintain serializability in transaction scheduling?
- What is a database view, and how does it differ from a table?
- What is the lifecycle of a transaction in a database, and what are its key stages?

Which of the following PL/SQL objects can be used to execute code automatically in response to an event in the database?

- A) Function
- B) Procedure
- C) Trigger
- D) Cursor

In PL/SQL, which of the following is used to store and manipulate query results row-by-row?

- A) Trigger
- B) Cursor
- C) View
- D) Procedure

A view in SQL can be best described as:

- A) A physical copy of a table
- B) A virtual table based on the result of a query
- C) A unique identifier for rows in a table
- D) A type of index for faster searches

Old Question Papers

- <http://www.aktuonline.com/papers/btech-cs-5-sem-data-base-management-system-rs501-2020.pdf>
- <http://www.aktuonline.com/papers/btech-cs-5-sem-database-management-system-KCS501-2018-19.pdf>
- <http://www.aktuonline.com/papers/btech-cs-5-sem-database-management-system-ncs-502-2017-18.pdf>
- <http://www.aktuonline.com/papers/btech-cs-5-sem-database-management-system-ncs-502-2016-17.pdf>

Expected Questions for University Exam

- Discuss the following terms (i) DDL Command (ii) DML command.
- What do you mean by Cursors in SQL?
- Explain the concept of Trigger in SQL.
- How do the ACID properties ensure data integrity in a DBMS?
- What is the concept of serializability in transaction scheduling? Explain conflict serializability and view serializability.
- How does DBMS ensure recoverability in non-serial schedules?
- Explain the concepts of cascading rollbacks and cascading aborts in transaction management.
- What are the two phases in 2PL and their significance?
- What is the difference between commit and rollback? How do they ensure data consistency?
- Compare lock-based concurrency control with timestamp-based concurrency control. What are the advantages and disadvantages of each?
- What is a serial schedule? How does it differ from a non-serial schedule?
- How does an index improve query performance?

Expected Questions for University Exam

- Write a PL/SQL Program to Add Two Numbers ii. Write PL/SQL Program for Fibonacci Series iii. Write PL/SQL Program to Find Greatest of Three Numbers
- What are the benefits of using synonyms in a DBMS?
- How can you modify or drop a sequence in DBMS?
- How do views help in data security and data abstraction?
- What is time zone management in DBMS, and why is it important?
- What are array functions in DBMS?
- How do synonyms help in managing database security and access control?
- Create the CUSTOMERS table having the following attributes: - (ID, NAME, AGE, ADDRESS, SALARY)
 - Insert ten records in customer table.
 - In Customer table delete those records which have age = 25 and then COMMIT the changes in the database.
 - In Customer table delete those records which have age = 30 and then Rollback the changes in the database.
 - Create three savepoint for customer table in that the three deletions have taken place.

Faculty Video Links, You tube & NPTEL Video Links and Online Courses Details

Video Links:

- https://www.youtube.com/watch?v=nfn3neARWWw&list=PL_uaekrhGzKtjZBDhKDHh9CBNZCv-GHY&index=31
- https://www.youtube.com/watch?v=TX4q1MrWsFk&list=PL_uaekrhGzKtjZBDhKDHh9CBNZCv-GHY&index=32
- https://www.youtube.com/watch?v=eUqLkCmGlKY&list=PL_uaekrhGzKtjZBDhKDHh9CBNZCv-GHY&index=33
- https://www.youtube.com/watch?v=JiaAgzXX30Y&list=PL_uaekrhGzKtjZBDhKDHh9CBNZCv-GHY&index=34
- https://www.youtube.com/watch?v=Ny-G949rf14&list=PL_uaekrhGzKtjZBDhKDHh9CBNZCv-GHY&index=35

References

- Thomas Cannolly and Carolyn Begg, “Database Systems: A Practical Approach to Design, Implementation and Management”,
- Pearson Education, 3rd Edition, 2007.
- Raghu Ramakrishnan and Johannes Gehrke “Database Management Systems”, McGraw-Hill, 3rd Edition, 2014.
- NoSQL and SQL Data Modeling: Bringing Together Data, Semantics, and Software, Ted Hills, 1st Edition, 2016.
- Brad Dayley “NoSQL with MongoDB in 24 Hours”, Sams Publisher, 1st Edition, 2014.

Thank You